

Advanced Manual Smart Contract Audit

August 1, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

Hoodroids

0x1245Fe8A6aB47AE083Dc2F494A84ae50039B672E

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	1	0	1	0
● Low-Risk	3	0	3	0
● Medium-Risk	5	0	5	0
● Critical-Risk	3	0	3	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way.
● Low-Risk	Won't cause any problems, but can be adjusted for improvement.
● Medium-Risk	Will likely cause problems and it is recommended to adjust.
● Critical-Risk	Will definitely cause problems, this needs to be adjusted.

Audit Summary

Project	Hoodroids
Website	https://hoodroids.io/
Blockchain	Robinhood
Source Code	https://robinhoodchain.blockscout.com/to-ken/0x1245Fe8A6aB47AE083Dc2F494A84ae50039B672E?tab=contract
Contract Address	0x1245Fe8A6aB47AE083Dc2F494A84ae50039B672E

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code. Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

Slither: Solidity static analysis framework
Remix: IDE Developer Tool
CWE: Common Weakness Enumeration
SWC: Smart Contract Weakness Classification and Test Cases
DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Audit Summary	3
Audit Scope	4
Detailed Findings	
ERC20 self-transfer bypasses balance check	6
Reentrancy via external registry calls	7
DoS risk: registry failure blocks transfers	8
approve() ambiguity between ERC20 and identity	9
Unbounded loops can make transfers revert	10
syncIdentities() is publicly callable (griefing)	11
Centralization: owner can change metadata/branding	12
Whitelist is powerful and owner-controlled	13
Non-standard ERC20/ERC721 interface behavior	14
Identity IDs can exceed NATIVE_SUPPLY	15
Agent URI JSON not escaped (injection risk)	16
Unused variable baseAgentURI	17
Disclaimer	18

● ERC20 self-transfer bypasses balance check

Critical-Risk - Will definitely cause problems, this needs to be adjusted.

Error Code	Description
AUD-001: Acknowledged	When <code>from == to</code> , <code>_transferERC20</code> does not subtract <code>amount</code> from the sender balance (it computes <code>senderBefore - amount</code> but discards it). This lets a user call <code>transfer(self, hugeAmount)</code> without reverting, emitting a misleading Transfer event and skipping mint/burn syncing. Integrations may treat it as a real transfer.

Code Snippet

```
if (from == to) {  
    senderBefore - amount;  
    emit ERC20Transfer(from, to, amount);  
    return;  
}
```

Recommendation

Remove the special-case or enforce `require(senderBefore >= amount)` and update balances consistently. For self-transfer, either do nothing and emit 0, or perform the same checks as normal transfers to avoid fake events.

Note

See team's response in attached file after audit report.

● Reentrancy via external registry calls

Critical-Risk - Will definitely cause problems, this needs to be adjusted.

Error Code	Description
AUD-002: Acknowledged	Mint/burn/transfer paths call the external `agentRegistry` (`register`, `setAgentURI`, `setMetadata`). If the registry is malicious or upgradeable, it can reenter into this contract (e.g., call `approve`, view functions, or other non-guarded functions) and cause unexpected state interactions/DoS. NonReentrant helps on some entrypoints but not all internal call graphs.

Code Snippet

```
_registerAgent(id, to);  
...  
IERC8004IdentityRegistry(agentRegistry).register();  
IERC8004IdentityRegistry(agentRegistry).setMetadata(...);
```

Recommendation

Treat the registry as untrusted: minimize external calls during state changes, consider a pull-based sync, or add stronger reentrancy protections around any function that can be reentered (including `approve`, admin setters if needed). Prefer immutable, audited registries.

Note

See team's response in attached file after audit report.

● DoS risk: registry failure blocks transfers

Critical-Risk - Will definitely cause problems, this needs to be adjusted.

Error Code	Description
AUD-003: Acknowledged	Transfers/mints/burns can revert if `agentRegistry` reverts or becomes unavailable, because `_registerAgent` and `_syncAgentHolder` are called inside token movement. This can freeze ERC20/ERC721-like transfers for all users if the registry is paused, upgraded badly, or rate-limited.

Code Snippet

```
_syncAgentHolder(id, to);  
...  
IERC8004IdentityRegistry(agentRegistry).setAgentURI(agentId, currentAgentURI);
```

Recommendation

Decouple token transfers from registry updates. Store pending updates and allow anyone to sync later, or wrap registry calls in `try/catch` with a non-blocking fallback while emitting an event for offchain repair.

Note

See team's response in attached file after audit report.

● approve() ambiguity between ERC20 and identity

Medium-Risk - Will likely cause problems and it is recommended to adjust.

Error Code	Description
AUD-004: Acknowledged	<code>approve(spender, amountOrId)</code> treats values <code>1..minted</code> as an identity approval, otherwise as ERC20 allowance. Users/integrations can accidentally approve an NFT-like identity when intending to approve a small ERC20 amount (or vice versa), causing unexpected permissions and potential asset loss.

Code Snippet

```
if (amountOrId > 0 && amountOrId <= minted) {  
    getApproved[amountOrId] = spender;  
} else {  
    allowance[msg.sender][spender] = amountOrId;  
}
```

Recommendation

Separate ERC20 and ERC721 approval functions (e.g., `approveERC20`, `approveIdentity`) or follow standard interfaces. At minimum, document clearly and add safeguards (like requiring a flag or using different function selectors).

Note

See team's response in attached file after audit report.

● Unbounded loops can make transfers revert

Medium-Risk - Will likely cause problems and it is recommended to adjust.

Error Code	Description
AUD-005: Acknowledged	ERC20 transfers can mint/burn multiple identities in loops based on whole-token changes. Large transfers to/from non-whitelisted accounts can exceed gas limits and revert, creating a denial-of-service for big holders or contracts that aggregate balances.

Code Snippet

```
for (uint256 i; i < burnCount; ++i) { _burnIdentity(from); }  
for (uint256 i; i < mintCount; ++i) { _mintIdentity(to); }
```

Recommendation

Add caps/batching (e.g., max identities per tx), or move identity sync to an explicit `syncIdentities` call only. Consider lazy mint/burn with checkpoints rather than per-unit loops.

Note

See team's response in attached file after audit report.

● syncIdentities() is publicly callable (griefing)

Medium-Risk - Will likely cause problems and it is recommended to adjust.

Error Code	Description
AUD-006: Acknowledged	Anyone can call `syncIdentities(account)` for any account. While it aims to reconcile state, it can be used to force expensive mint/burn loops on a victim account (gas griefing) or to trigger registry calls that may revert and waste gas.

Code Snippet

```
function syncIdentities(address account) external nonReentrant {  
    ...  
    for (...) { _mintIdentity(account); }  
    for (...) { _burnIdentity(account); }  
}
```

Recommendation

Restrict to `account` or approved operators, or add rate limits / max operations per call. Alternatively, make it a view-only helper and require the account to opt-in to syncing.

Note

See team's response in attached file after audit report.

● Centralization: owner can change metadata/branding

Medium-Risk - Will likely cause problems and it is recommended to adjust.

Error Code	Description
AUD-007: Acknowledged	Owner can change `baseTokenURI`, `baseAgentURI`, `contractMetadataURI`, `name`, and `symbol` at any time. This allows altering what users see as the asset/agent identity, enabling rug-style metadata swaps or misleading branding changes.

Code Snippet

```
function setTokenURI(...) public onlyOwner { baseTokenURI = newBaseURI; }  
function setNameSymbol(...) public onlyOwner { name = newName; symbol = newSymbol; }
```

Recommendation

Use timelocks/multisig for admin actions, emit clear events (already present), and consider permanently freezing URIs and name/symbol after launch via an irreversible `freeze()`.

Note

See team's response in attached file after audit report.

● Whitelist is powerful and owner-controlled

Medium-Risk - Will likely cause problems and it is recommended to adjust.

Error Code	Description
AUD-008: Acknowledged	Whitelisted addresses skip identity mint/burn during ERC20 transfers. Owner can whitelist/unwhitelist any address, changing how balances map to identities and potentially breaking user expectations, marketplace logic, or fairness assumptions.

Code Snippet

```
mapping(address target => bool state) public whitelist;  
function setWhitelist(address target, bool state) public onlyOwner { whitelist[target] = state; }
```

Recommendation

Limit whitelist usage (e.g., only known system contracts), add a timelock, and publish a policy. Consider making whitelist immutable after deployment or only allowing additions (no removals).

Note

See team's response in attached file after audit report.

● Non-standard ERC20/ERC721 interface behavior

Low-Risk - Won't cause any problems, but can be adjusted for improvement.

Error Code	Description
AUD-009: Acknowledged	The contract emits ERC20-like events but does not implement full ERC20 (no `Transfer` event name, no `Approval` event for ERC20, no `totalSupply()` function signature, etc.) and mixes ERC721 semantics. Many wallets/exchanges may mis-handle approvals/transfers, leading to user mistakes.

Code Snippet

```
event ERC20Transfer(...);  
event ERC20Approval(...);  
function transferFrom(..., uint256 amountOrId) ...
```

Recommendation

Implement standard ERC20 and ERC721 interfaces explicitly (IERC20/IERC721) or clearly brand as non-standard. Provide separate methods and events matching standards to reduce integration risk.

Note

See team's response in attached file after audit report.

● Identity IDs can exceed NATIVE_SUPPLY

Low-Risk - Won't cause any problems, but can be adjusted for improvement.

Error Code	Description
AUD-010: Acknowledged	<code>`minted`</code> increases monotonically and can exceed <code>`NATIVE_SUPPLY`</code> over time due to burn/remint cycles, while <code>`tokenURI`</code> maps IDs modulo <code>`NATIVE_SUPPLY`</code> . This causes multiple identities to share the same metadata file, which may confuse users and marketplaces.

Code Snippet

```
uint256 metadataId = ((id - 1) % NATIVE_SUPPLY) + 1;  
return string.concat(baseTokenURI, metadataId.toString(), ".json");
```

Recommendation

If uniqueness is expected, make metadata depend on ``id`` directly or cap ``minted`` by reusing burned IDs. Document clearly that metadata is cyclic if that is intended.

Note

See team's response in attached file after audit report.

● Agent URI JSON not escaped (injection risk)

Low-Risk - Won't cause any problems, but can be adjusted for improvement.

Error Code	Description
AUD-011: Acknowledged	JSON is built via string concatenation using `name`/`symbol` and other fields without escaping quotes/special characters. Owner can set a name/symbol that breaks JSON or injects unexpected fields, confusing indexers and UIs.

Code Snippet

```
string.concat('{"name":"","symbol":"","token":' + ERC8004 + ' Agent #', ...)
```

Recommendation

Escape JSON strings (replace `",`, `\",` control chars) or restrict allowed characters for `name`/`symbol`. Consider generating metadata offchain or using a robust JSON builder.

Note

See team's response in attached file after audit report.

• Unused variable baseAgentURI

Informational - Does not compromise the functionality of the contract in any way.

Error Code	Description
AUD-012: Acknowledged	`baseAgentURI` is settable but not used in `agentURI()` (which always returns onchain base64 JSON). This may be a design oversight and can mislead operators who think changing it affects agent metadata.

Code Snippet

```
string public baseAgentURI;  
function setAgentURI(...) public onlyOwner { baseAgentURI = newBaseAgentURI; }
```

Recommendation

Either use `baseAgentURI` in `agentURI()` (e.g., as an external base) or remove it to avoid confusion. If kept, document its purpose clearly.

Note

See team's response in attached file after audit report.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.

Response to Coinsult Findings

Finding-by-finding response for the deployed Hoodroids ERC8404 token, covering exploitability, source safeguards, test evidence, current onchain conditions, and requested classification.

REVIEWED DEPLOYMENT

0x1245Fe8A6aB47AE083Dc2F494A84ae50039B672E

Robinhood Chain · ERC8404AgentToken · Hoodroids (HOODROIDS)

Scope and Snapshot

This document responds to findings AUD-001 through AUD-012. It distinguishes source-level design limitations from conditions that are currently executable on the deployed contract. Test success demonstrates the tested scenarios; it does not guarantee future external-registry availability or unlimited gas capacity.

SNAPSHOT BLOCK 25,049,265	TOKEN OWNER address(0)	MINTED IDS 1,179
LIVE IDENTITIES 602 / 1,000	REGISTRY VERSION 2.0.0	REGISTRY OWNER 0x28c85dBEbe5c981b8a69721CB7f3421e8106F5B6

Identity registry: 0xc41b667b6f9c798ec05f3e16991b80adbc58d684

Registry implementation: 0x6c9eee32305afe2d683850194d9295706330d51b

Test evidence used in this response

- forge test --match-contract 'ERC8404AgentTokenTest|ERC8404AuditResponseTest' -vv: 50 passed, 0 failed.
- Robinhood fork with recreated pool and mock registry: 30 boundary-crossing swaps, 30 identities and 30 agents minted.
- Average swap call gas in that test: 2, 298, 091.
- Targeted audit regression tests: test/ERC8404AuditResponse.t.sol.

Classification Summary

ID	FINDING	REQUESTED CLASSIFICATION
AUD-001	ERC20 self-transfer balance check	False positive
AUD-002	Registry-call reentrancy	Downgrade to Low
AUD-003	Registry failure and transfer liveness	Downgrade to Medium
AUD-004	Mixed approve dispatch	Design limitation
AUD-005	Unbounded identity loops	Acknowledged, Medium unchanged
AUD-006	Permissionless identity synchronization	False positive
AUD-007	Owner-controlled metadata	Downgrade to Informational
AUD-008	Owner-controlled whitelist	Downgrade to Low
AUD-009	Non-standard hybrid behavior	Design limitation
AUD-010	Identity IDs beyond native supply	Design limitation
AUD-011	Agent JSON escaping	Downgrade to Informational
AUD-012	Unused baseAgentURI	Informational unchanged

AUD-001

ERC20 self-transfer bypasses balance check

1. POSITION	Disagree.
2. EXPLOITABILITY	<code>senderBefore</code> - amount is still evaluated under Solidity 0.8 checked arithmetic. An amount above balance reverts with <code>Panic(0x11)</code> . A valid self-transfer has zero net balance and identity effect.
3. SAFEGUARDS	<code>_transferERC20</code> rejects zero addresses and performs the checked subtraction before assigning the receiver balance.
4. TESTS	<code>testERC20SelfTransferDoesNotChangeBalanceOrIdentities</code> and <code>testAUD0010oversizedSelfTransferReverts</code> . A live <code>eth_call</code> from a zero-balance account also reverted with <code>Panic(0x11)</code> .
5. ONCHAIN STATE	The deployed bytecode uses Solidity 0.8 checked arithmetic; no owner-controlled bypass remains.
6. REQUEST	False positive

AUD-002

Reentrancy via external registry calls

1. POSITION	Partially agree that external calls exist; disagree that a Critical asset-loss path is demonstrated.
2. EXPLOITABILITY	All asset-moving entry points share one reentrancy guard. Reentry into approvals can only create permissions belonging to the registry as <code>msg.sender</code> , not a holder.
3. SAFEGUARDS	<code>transfer</code> , <code>transferFrom</code> , both <code>safeTransferFrom</code> overloads, and <code>syncIdentities</code> are <code>nonReentrant</code> . <code>onERC721Received</code> accepts only the immutable registry.
4. TESTS	<code>testAUD002RegistryCannotReenterAssetTransfer</code> confirms registry reentry fails with <code>ReentrancyGuardReentrantCall()</code> while the outer state remains consistent.
5. ONCHAIN STATE	The token owner is zero. The live registry is version 2.0.0, but its proxy remains upgradeable by its separate owner.
6. REQUEST	Downgrade to Low

AUD-003

Registry failure blocks transfers

1. POSITION	Agree in part.
2. EXPLOITABILITY	A reverting registry blocks transfers that mint, burn, or synchronize an identity. It does not block a fractional transfer that crosses no whole-token boundary. Reverts are atomic, so balances cannot be partially moved or lost.
3. SAFEGUARDS	The registry is immutable. Whitelisting and whole-boundary checks reduce calls. There is intentionally no try/catch fallback, so registry integrity and token state cannot silently diverge.
4. TESTS	testAUD003RegistryFailureBlocksOnlyBoundaryCrossingTransfer proves a 0.5-token non-boundary transfer succeeds while the next boundary-crossing transfer reverts and rolls back.
5. ONCHAIN STATE	The live registry currently responds and exposes no pause/rate-limit surface, but its proxy is upgradeable. Tests cannot guarantee future availability.
6. REQUEST	Downgrade to Medium

AUD-004

approve ambiguity between ERC20 and identity

1. POSITION	Agree; intentional ERC404-style dispatch.
2. EXPLOITABILITY	Values 1..minted are ambiguous. An identity approval still requires ownership or operator authorization; otherwise the call reverts. Normal 18-decimal token allowances are generally above this low-wei range.
3. SAFEGUARDS	The identity branch checks ownerOf and caller authorization. The fungible branch writes only the caller's ERC20 allowance.
4. TESTS	testERC721ApprovalWorks, testERC20AllowanceTransferWorks, and testMixedTransferFromTreatsActiveIdAsNftAndLargeValueAsERC20Amount.
5. ONCHAIN STATE	At the snapshot, the ambiguous range is 1..1,179 wei.
6. REQUEST	Design limitation

AUD-005

Unbounded loops can make transfers revert

1. POSITION	Agree.
2. EXPLOITABILITY	A large whole-balance change can exceed practical block gas. The complete transaction reverts atomically; users must split very large transfers. This is liveness risk, not partial fund loss.
3. SAFEGUARDS	<code>liveIdentities</code> is capped at <code>NATIVE_SUPPLY == 1,000</code> . System addresses are whitelisted to avoid identity loops while assets sit in DEX infrastructure.
4. TESTS	<code>testCannotMintMoreThanOneThousandLiveIdentities</code> . The 30-wallet fork test averaged 2,298,091 gas per separate one-identity buy, supporting rather than disproving the finding.
5. ONCHAIN STATE	<code>liveIdentities == 602</code> at the snapshot. Ownership is renounced, so loop behavior cannot be upgraded in place.
6. REQUEST	Acknowledged - Medium unchanged

AUD-006

Public `syncIdentities` griefing

1. POSITION	Disagree with the stated victim gas-griefing scenario.
2. EXPLOITABILITY	The caller pays gas. The function can only reconcile identity count to <code>balanceOf(account) / unit</code> ; it cannot select an arbitrary count or move funds. Repeated calls after reconciliation perform no registry write.
3. SAFEGUARDS	<code>syncIdentities</code> is nonReentrant, rejects zero address, returns for whitelisted accounts, and loops only over the count delta.
4. TESTS	<code>testAUD006RepeatedSyncHasNoWorkAfterReconciliation</code> and <code>testSyncIdentitiesBringsNonWhitelistedAccountBackIntoLine</code> .
5. ONCHAIN STATE	The whitelist is frozen by token ownership renouncement.
6. REQUEST	False positive for griefing; permissionless reconciliation is intentional.

AUD-007

Owner can change metadata and branding

1. POSITION	Agree for owner-controlled deployments; not executable on this deployment.
2. EXPLOITABILITY	Every listed setter is onlyOwner, and the deployed owner is permanently address(0).
3. SAFEGUARDS	OpenZeppelin Ownable gates setTokenURI, setAgentURI, setContractURI, and setNameSymbol.
4. TESTS	testNonOwnerCannotUpdateMetadata.
5. ONCHAIN STATE	Name, symbol, published IPFS base, empty baseAgentURI, and collection URI are frozen.
6. REQUEST	Downgrade to Informational

AUD-008

Powerful owner-controlled whitelist

1. POSITION	Agree historically; ongoing owner control has been removed.
2. EXPLOITABILITY	No entry can now be added or removed. Existing exemptions remain a fixed configuration and affect identity synchronization, not fungible balances.
3. SAFEGUARDS	setWhitelist is onlyOwner and emits whitelistUpdated.
4. TESTS	testOnlyOwnerCanUpdateWhitelist, testWhitelistedReceiverDoesNotMintIdentitiesOrAgents, and testWhitelistedSenderDoesNotBurnIdentities.
5. ONCHAIN STATE	Thirteen frozen entries cover deployer, V4 infrastructure, Permit2, hook/collector, UNCX V4 locker, and official V2/V3 infrastructure. At snapshot, deployer held 0.6555114449 token and V4 PoolManager held 330.3726970006; other listed infrastructure held zero.
6. REQUEST	Downgrade to Low

AUD-009

Non-standard ERC20/ERC721 behavior

1. POSITION	Partially agree. The report's claim that <code>totalSupply()</code> is absent is incorrect.
2. EXPLOITABILITY	<code>public immutable totalSupply</code> generates the selector. However, ERC20 events are deliberately separate, <code>balanceOf</code> is fungible balance, and identity count uses <code>erc721BalanceOf</code> . Universal ERC20/ERC721 compatibility is not claimed.
3. SAFEGUARDS	Standard ERC721 identity Transfer events and ERC165 interface responses are present; fungible activity emits explicit ERC20Transfer/ERC20Approval.
4. TESTS	<code>testConstructorSetsInitialState</code> , <code>testSupportsOpenSeaERC721Interfaces</code> , <code>testEmitERC20TransferEvent</code> , and standard identity-event tests.
5. ONCHAIN STATE	Current marketplace and DEX operation demonstrates specific integrations, not universal compatibility.
6. REQUEST	Design limitation ; remove the inaccurate <code>totalSupply()</code> subclaim.

AUD-010

Identity IDs can exceed `NATIVE_SUPPLY`

1. POSITION	Agree; intentional cyclic collection metadata.
2. EXPLOITABILITY	Identity and agent IDs remain unique and are never reassigned. Visual metadata repeats modulo 1,000. This cannot collide ownership records or move funds, but marketplaces may show repeated artwork and traits.
3. SAFEGUARDS	<code>liveIdentities</code> is capped at 1,000, <code>minted</code> is monotonic, and bidirectional identity/agent mappings remain unique.
4. TESTS	<code>testBurnedIdsAreNotReassignedAndNewBuysRegisterNewAgents</code> , mapping-bijection tests, and <code>testTokenURIWrapsMetadataIdIntoOneThousandRange</code> .
5. ONCHAIN STATE	<code>minted == 1,179</code> and <code>liveIdentities == 602</code> , confirming that historical IDs exceed 1,000 while live supply does not.
6. REQUEST	Design limitation

AUD-011

Agent URI JSON is not escaped

1. POSITION	Agree at source level.
2. EXPLOITABILITY	Deployed owner-controlled strings are frozen and contain no JSON control characters. Numeric values, statuses, and addresses use contract-generated fixed formats, so the described owner-driven injection is no longer executable.
3. SAFEGUARDS	Current mitigation is frozen safe input after ownership renouncement. There is no general JSON-escaping helper.
4. TESTS	testAgentURIIsPublicDataJSON and metadata synchronization tests validate current structure. No general escaping fuzz test is claimed.
5. ONCHAIN STATE	Current name, symbol, and URI values are safe ASCII strings and cannot be changed.
6. REQUEST	Downgrade to Informational ; source-level hardening is acknowledged.

AUD-012

Unused baseAgentURI

1. POSITION	Agree.
2. EXPLOITABILITY	agentURI() always returns onchain Base64 JSON and does not read baseAgentURI. This has no security, balance, or authorization effect.
3. SAFEGUARDS	Not applicable; this is dead configuration state rather than an exploitable path.
4. TESTS	testSetTokenURIAndAgentURIUpdateMetadataBases confirms the variable changes in owner-controlled tests while agentURI() remains a data URI.
5. ONCHAIN STATE	baseAgentURI is empty and ownership is renounced.
6. REQUEST	Informational unchanged

Qualification. ERC8404 is an experimental hybrid derived from ERC404-style fungible/non-fungible mechanics and ERC-8004 agent registration. The deployed safeguards and tests reduce demonstrated risk in the scenarios above, but they do not convert design limitations into formal ERC20/ERC721 compliance or guarantee the future behavior of an independently upgradeable registry.